Enhancing Defect Tracking Systems to Facilitate Software Quality Improvement

Jingyue Li, DNV Research and Innovation

Tor Stålhane and Reidar Conradi, Norwegian University of Science and Technology

Jan M.W. Kristiansen, Steria

// Simple goal-oriented changes to existing data in defect tracking systems provides valuable and prompt information to improve software quality assessment and assurance. //

SOFTWARE COMPANIES USUALLY apply data from defect tracking systems (DTSs) to ensure that reported defects eventually get fixed. Such data has obvious potential for use in current software quality assessment (SQA) and in planning future software process improvement (SPI) initiatives.[1] However, in examinations of nine Norwegian companies' DTSs, we found that most of the data entered in these systems was never used, irrelevant, unreliable, or difficult to apply to SQA and SPI.

In response to our findings, we worked with two of the companies to improve their DTSs with a view to facilitating SQA and SPI. The improvements complied with the goal/question/metric (GQM) paradigm.[2] We focused mainly on either revising the values of existing defect classification attributes in an existing DTS or introducing new attributes. Primarily, we wanted to give project managers and developers more current, relevant, correct, and easy-to-analyze defect data for assessing software quality and finding potential SPI measures in a cost-effective way.

We evaluated the improved DTS support in two rounds of defect analyses: one to initialize SPI and the other to assess the effectiveness of the SPI activities. The results showed that new SPI activities in both companies significantly reduced the defect densities and increased the efficiency of fixing the remaining defects. Lessons learned from this study illustrate how to keep developers and testers motivated to enter high-quality defect data into their DTSs. The study also reveals several pitfalls that typically reduce the reported data's quality.

DTS Data from the Investigation

We studied nine companies' DTSs. All the companies (except one that had fewer than 100 employees) used at least 10 attributes to record defects—for example, textual summary, detailed description, priority, severity, and calendar dates of fixes. Table 1 indicates which defect attributes each company included in its DTS.

Some of the defect data was ready for SQA or SPI. For example, all companies recorded the date and time that they created a defect report. Six of the nine companies used a dedicated attribute to record the email address or name of the person who created the

0740-7459/12/$31.00 © 2012 IEEE

TABLE 1

Defect attributes in the examined tracking systems.

Content/attributes of defect reports and log of defect fixes		Company identifiers and number of employees								
		AN (320)	CO (180)	CS (92,000)	PW (500)	DP (6,000)	SN (400)	DT (9,000)	SA (30,000)	DA (10)
Description	Defect report ID	X	X		X	X	X	X	X	
	Short textual summary				X	X	X	X	X	
	Detailed description	X	X	X	X			X		X
	High-level category*	X	X	X	X		X	X	X	
Time stamp and persons involved	Created date and time	X	X	X	X	X		X	X	
	Creator and contact info	X	X	X	X		X	X	X	
	Modified date and time	X	X	X	X	X		X	X	
	Modified by	X				X	X			
	Responsible person		X	X	X		X	X		
	Deadline to finish the defect fix				X		X			
	Closed time	X			X					X
	Estimated duration to fix				X			X	X	
Impact	Priority	X	X		X	X	X	X	X	
	Severity	X	X	X	X	X		X	X	
Status trace	Status	X	X		X	X	X	X		
	Resolution				X		X			
	New release no. after fix	X			X					

*High-level categories: defect, enhancement, duplication, or no defect (that is, wrong report/not a defect)

original report. Seven companies assigned a severity value to each defect. By combing such data, a company can quickly find critical quality issues, such as severe defects that important customers reported after a release.

In addition, seven companies recorded the name of the infected software "modules." This information can help developers identify a system's most defect-prone or change-prone parts. Finding ways to eliminate these "hot" parts of the system can help companies maximize their return on investment (ROI).

Information in a company's defect-fixing work logs can also indicate what to improve to speed up the process—for example, developers' complaints about a complex software architecture suggest the need to adjust the design.

However, the studied companies used only some of their defect data—for example, tracking defect status—for project management. None had

TABLE 1 (CONT'D)

Defect attributes in the examined tracking systems.

Content/attributes of defect reports and log of defect fixes		Company identifiers and number of employees								
		AN (320)	CO (180)	CS (92,000)	PW (500)	DP (6,000)	SN (400)	DT (9,000)	SA (30,000)	DA (10)
Test activity	Tester					X		X	X	
	Test case ID		X		X	X				
	Test priority		X							
	Test description		X		X	X		X	X	
Location of defects	Release	X			X	X		X	X	
	Module(s)	X	X	X	X		X	X	X	
	Version		X	X	X		X			
	Operating system and hardware				X					
Supplementary information	Comments	X	X		X			X		X
	Related link				X		X	X		
	Work log for defect fixing activities			X	X				X	

*High-level categories: defect, enhancement, duplication, or no defect (i.e., wrong report/not a defect)

analyzed the SQA- and SPI-related data in their DTSs. The assembled information behaved largely as an information graveyard. Furthermore, because the DTSs were conceived without explicit SQA and SPI goals in mind, the existing DTS data was usually inadequate for these purposes. Instead, most companies were simply satisfied that a defect was somewhat fixed and didn't track how much effort was spent doing so or why the defect occurred in the first place.

None of the companies recorded the actual effort used to fix a defect, although some reported some aspect of the duration. In either case, they had little information available to measure the cost-effectiveness of the defect fix or to perform root-cause analysis to prevent further defects, especially for those that were most costly to fix.

Other problems included

- *incomplete data*. More than 20 percent of the data hadn't been filled in for defect attributes such as severity and location.
- *inconsistent data*. Some people used the name of an embedding module or subsystem for a defect's location, while others used a function name.
- *mixed data*. As Table 1 shows, four companies didn't define a separate attribute to indicate how a defect was discovered. Instead, they included this information with other text in the short summary or detailed description of the defect, making it difficult to extract testing-related information for SQA or SPI purposes.

So even defect data potentially available in the existing DTS was inconsistent and difficult to find.

Two Case Studies to Improve the DTS

We helped two companies from our study improve their DTSs by following the GQM paradigm to revise and introduce a defect classification scheme.[4,5]

Company DP

The first case, company DP, is a software house that builds business-critical systems, primarily for the financial sector. Here, different departments used the existing DTS in different ways, and because the data wasn't systematically useful, there were few incentives to improve either the system or its use.

However, we performed a gap analysis, which showed that the company's defect reporting and prioritization process was a main concern of developers and testers. Reducing the effort to fix defects was another main concern.

Goals, questions, and metrics. The DTS improvement aimed to reduce the defect density and to improve defect-fixing efficiency. To achieve this goal, we wanted the DTS to provide supplementary information that the quality assurance (QA) managers could use to answer the following questions:

- What are the main defect types?
- What can the company do to prevent defects in a project's early stages?
- What are the reasons for the actual defect-fixing effort?

The existing DTS wasn't instrumented to collect data for answering these questions. We proposed revisions based on both analysis of existing data and QA managers' suggestions. To avoid abrupt changes, we introduced no new defect attributes, only revised values of existing ones.

Validation and follow-up. Together with the test manager, one developer, and one project manager, we performed two rounds of validation of the proposed DTS revision. We classified defects from earlier projects, using the proposed DTS to check whether the revised attributes fit the company's context and its SQA and SPI purposes.

Company DP's existing DTS wasn't instrumented to collect data for GQM questions. We proposed revisions.

Several attributes improved in company DP's DTS:

- *Fixing type*. A new set of values categorized developers' defect-fixing activities.
- *Effort*. Three qualitative values classified a defect-fixing effort: *simple* meant that developers would spend less than 20 minutes total effort to reproduce, analyze, and fix a defect; *medium* meant the effort would take between 20 minutes and 4 hours; and *extensive* meant the effort would take more than 4 hours. (We used this simplified Likert scale because asking developers to expend the effort to provide a more precise number for past events wasn't cost effective and didn't benefit our intended analysis.)
- *Root cause*. Project entities such as requirements, design, development, and documentation characterized each defect's origin.

After the validation, we gave a presentation to developers, testers, and project managers to explain how the company could use the revised attributes. The company also revised the DTS workflow to remind developers and testers to fill in defect data before closing a defect.

Company PW

The second case, company PW, is a software product-line company with only one product, which it deploys on more than 50 different operating systems and hardware platforms. A gap analysis, similar to the one for company DP, showed that QA personnel prioritized a more formal DTS as a main concern. QA managers wanted a mechanism to analyze defect information quickly—first, because the company receives thousands of defect reports every month and, second, because the external release cycle is about three months.

Goals, questions, and metrics. Company PW also aimed to reduce defect density and to improve defect-fixing efficiency. The DTS needed to provide information to answer the following questions:

- What can the company do to prevent defects in the early project stages and to detect them before the new software release reaches customers?
- Which testing activities discovered or reproduced the most defects?
- What are the reasons for the actual defect-fixing effort?

We added or revised defect attributes according to the IBM Orthogonal Defect Classification (ODC),[4] the "suspected cause" attribute of the IEEE Standard Classification for Software Anomalies,[5] and suggestions from the company's QA managers.

Validation and follow-up. Also in this case, we performed two rounds of validation and, with one QA manager, one tester, and one developer, tried to reclassify defects that were reported in previous projects. Added or revised attributes for this company's DTS, after validation, included

- *Effort*. Two qualitative values—*quick-fix* and *time-consuming*—classified a defect-fixing effort; the latter means spending more than one total person-day to reproduce, analyze, and fix the defect. We used two categories rather than three as in company DP, because we wanted just to pick out those costly defects and focus on them.
- *Fixing type*. Values combined the extension of the IBM ODC "type" attributes[4] and the categories of the company's typical defect-fixing activities.
- *Severity*. Values defined a defect's impact on the software's functionality and the user's experience.

- *Trigger*. Values represented the company's typical testing activities.
- *Root cause*. Values characterized each defect's origin in project entities such as requirements, design, development, and documentation.

After validations, we presented the added or revised attributes to project managers. We uploaded a revised online manual to help DTS users. To avoid making large changes in the system, we separated the newly added attributes from the existing ones and called them "extra" attributes.

Software Quality Insights from Improved DTSs

The data collected through the improved DTS provided valuable information to the two companies to support their SPI. We performed two rounds of large-scale defect analysis on the newly collected DTS data after the companies launched the improved DTSs. We used the first-round analysis to discover software process weaknesses and to initiate the SPI. After the companies had performed the SPI for a while, we performed the second-round analysis to assess the SPI's effectiveness.

Company DP: Supplementing Earlier Analysis Data

In the first-round defect analysis for company DP, we downloaded information from 1,053 defects reported during system tests in two releases of a large system. Analyzing the root-cause and fixing-type attributes showed that 397 of them related to development and were responsible for the majority of defect-fixing efforts. Most of these 397 defects comprised wrong or missing functionality or the display of incorrect or missing text messages to users.

When the QA manager saw the analysis results, she explained that those defects were probably the result of hiring a large number of consultants who had excellent development and coding experience but insufficient banking domain knowledge. Without the defect data and analysis, the QA manager wouldn't have acquired this insight, especially in light of an early post-mortem analysis showing that company DP's developers were proud of their application domain knowledge. They preferred high-level requirements specifications that let them use their creativity in design and coding.

In response to the defect analysis results, the company changed its hiring strategy by putting more emphasis on evaluating domain knowledge before recruiting new staff. Six months later, we collected new defect data of the same system's follow-up releases and did a second-round analysis of the effort spent on fixing defects. To compare the new data with the data collected before the hiring strategy change, we quantified the three qualitative categories (simple, medium, and extensive) by assigning them values (10 minutes, 1 hour, and 11 hours, respectively). The share of effort spent on fixing defects attributable to missing domain knowledge dropped from 60 to 30 percent. The effort for fixing all defect types decreased by 25 percent.

Company PW: Supporting SPI Decisions

In the first-round defect analysis of company PW, we downloaded and analyzed 796 defects from two projects. The developers had classified 166 of these defects as time-consuming. Simple statistical analyses of the fixing-type attribute showed that more thorough code reviews could easily detect 60 percent of these time-consuming defects—for example, the defects related to wrong algorithms or missing exception checking and handling.

One project manager from these projects had felt the need for more formal code and design reviews, but he had no data to justify the extra effort. After seeing the defect analysis results, as a first step, he required the project developers to perform formal code reviews after each defect fix.

> The improved DTS provided valuable information to initialize and justify software process improvements.

As with company DP, we collected newly reported defect data for the project—this time, 12 months after the new code reviews were enforced—and compared it with the data we collected previously. Results showed that the share of post-release defects attributable to faulty defect fixes had dropped from 33 to 8 percent.

Lessons Learned from Data Collection

Data-driven SPI decisions require high-quality DTS data. Our study reveals several issues regarding its collection.

Lean, Goal-Oriented Data Collection

Before proposing a DTS improvement, company managers should have a clear goal of what analyses they want to perform and why. Following the GQM spirit of lean and relevant data, we gathered only the minimum data for the intended extra analyses. For example, we used only three qualitative values for categorizing a defect-fixing effort and saved developers from having to fill in accurate numbers because our focus was on identifying and preventing the "time-consuming" defects, not on doing a full ROI analysis.

Motivating Users

One major issue in improving DTSs is the pressure of meeting delivery deadlines, which makes developers believe they don't have time to fill in new defect data attributes. DTS users must be convinced that the data collection is in their interest and won't take much time.

Prior to the improvement project. In company DP, although the managers initiated the idea of improving their DTS, we gave a presentation to developers and testers to explain the reason for changing the DTS categories and the possible benefits of gathering and analyzing the data before deploying the improved DTS. Before the first-round large-scale analysis of new data from the improved DTS, we performed several rounds of small-scale analyses on some preliminary defect data and fed all the preliminary analysis results back to managers, who presented and discussed them with their staff. In this way, misunderstandings and comments from managers or developers were dealt with before collecting new data.

Company PW involved mainly project managers in initiation, design, validation, and training of the improved DTS because the top managers didn't want to involve developers and testers too much before they saw significant benefits. The top managers expected project managers to explain the improved DTS to developers or testers when they asked them to fill in the new data. Before the first-round large-scale defect analysis on the newly collected DTS data, the company had performed no preliminary defect analysis similar to the analysis for company DP because top managers were concerned that no statistically significant results could be achieved without a large amount of data.

When we performed the first-round large-scale defect analysis on the newly collected DTS data, we found that missing or inconsistent data happened much more frequently in company PW than in company DP. Additionally, developers and testers in company PW were less positive toward the improved DTS. In the email survey to collect feedback on the DTS improvement after this round of defect analysis, several PW developers complained that they didn't fully understand the revised defect attributes and therefore didn't believe the improved DTS brought real value to their projects.

In response to this finding, we presented the first-round defect analysis results to 25 developers and testers in an internal PW workshop. Participant attitudes toward the changes improved, along with their willingness to fill in quality data.

In SPI feedback. Dieter Rombach and his colleagues showed that the SPI effort to prevent and discover defects early in the software life cycle paid off gradually through fewer subsequent defects.[7] In the projects we investigated, fixing a post-release defect in companies DP and PW took an average of 11 and 8 person-hours, respectively. Using DTS data to avoid just a few defects early in the project, especially those classified as time-consuming, will more than offset the extra effort spent collecting more defect data.

> DTS users must be convinced that the data collection is in their interest and won't take much time.

Our second round of defect analysis also showed that effort spent on proper SPI activities, which we derived from the DTS data analysis, paid off handsomely. Although it's theoretically and empirically easy to foresee improved ROI by improving the DTS, we had to convince developers and testers that the collected data will benefit them in their day-to-day work. DTS users need quick feedback to show them that the defect attributes and corresponding values are relevant, the work is doable, and their total efforts are beneficial. Slow data feedback leads either to little or low-quality data and developers' disrespect for SPI work in general.

Potential Pitfalls in Defect Data Quality

Although we performed two rounds of validation before launching the improved DTS, the first- and second-round analyses of newly reported defects in the improved DTS still revealed several pitfalls that DTS improvement projects must avoid.

First-round analyses. First, we found that developers would forget to reclassify a defect if it eventually involved more effort than they originally thought it would. For example, a defect goes through several states in company PW—from newly reported to confirmed to fixed and verified. In our improved DTS, we asked developers or testers to classify defects according to the effort spent when the defect is completely fixed and verified. However, from reading the defect work log, we found some defects that were initially classified as quick fixes, but were later reopened and refixed in a way that warranted reclassification as time-consuming. However, the defects weren't reclassified. DTSs therefore need a mechanism to remind developers and testers to refill or correct this data after they reexamine a defect.

Another potential pitfall occurs when default values bias results. For example, company PW's improved DTS presented some attribute values in a dropdown menu. The menu was set with a default value to illustrate the attribute's meaning. Analysis showed that more than 70 percent of the defects were categorized under this default value. By reading the work logs, we found that some of these values should have been different. We suspect that developers and testers simply skipped this attribute when they saw that the system already provided a default value. In retrospect, we believe DTSs should avoid using default attribute values.

We designed defect attribute values to be orthogonal, so we required users to make a single choice for each defect attribute. However, we found that multiple choice is sometimes more applicable (other research concurs[8]). For example, in company PW, we found that fixing a complex defect might include both correcting both a variable's assignment and the algorithm for using it. Classifying such a defect as either an assignment or an algorithm defect type will be incorrect. Thus, DTS designers should carefully examine whether the values of certain attributes should be single choice or multiple choice.

Second-round analyses. Timely updates of defect attributes and their corresponding values are important. When a company's practices change, the defect attribute values must follow suit.

When company PW asked developers to start performing more formal code review after they fixed a defect, the DTS needed updating to add the value "code review" in the "trigger" attributes, so developers could properly classify the review's outcome. Conversely, some attributes (or attribute values) might become outdated over time. In this case, the QA personnel responsible for the DTS needs to carefully remove the attributes, ensuring that they are really no longer relevant (and backing the decision up with data).

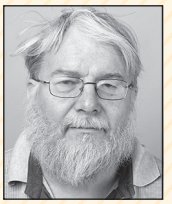
Lessons Learned from Root Cause Analysis

In company PW, we asked developers and testers to give their ideas of a defect's root cause. However, we found that they knew only what was happening in the code and not how to trace a defect's causes to earlier stages of a project. Thus, different people should probably specify the root-cause attribute from different perspectives; but who should they be and how can projects resolve conflicting proposals?

In company DP, the statistical analysis of the defect data said a lot about what was happening but provided limited information about why. In the first-round defect analysis, we had to identify root causes by combining statistical analysis of the defect data with information in the defect description

ABOUT THE AUTHORS

JINGYUE LI is a senior researcher at DNV Research & Innovation. His research interests include software process improvement, empirical software engineering, and software reliability. Li has a PhD in software engineering from the Norwegian University of Science and Technology. He's a member of IEEE and the ACM. Contact him at jingyue.li@dnv.com.



TOR STÅLHANE is a full professor of software engineering at the Norwegian University of Science and Technology (NTNU). His research interests include software reliability, software process improvement, and systems safety. Stålhane has a PhD in applied statistics from NTNU. Contact him at tor.stalhane@idi.ntnu.no.

REIDAR CONRADI is a full professor in the Department of Computer and Information Science at the Norwegian University of Science and Technology (NTNU). His research interests include software quality, software process improvement, version models, software evolution, component-based software engineering, open source software and related impacts, software engineering education, and associated empirical methods and studies. Conradi has a PhD in software engineering from NTNU. He's a member of IEEE, IFIP WG2.4, ACM, and the International Software Engineering Research Network. Contact him at conradi@idi.ntnu.no.

JAN M.W. KRISTIANSEN is a software engineer with Steria AS. His research interests include agile methods for software development, software process improvement, and open source software. Kristiansen has a master's degree in computer science from the Norwegian University of Science and Technology. Contact him at jmk@steria.no.

(free text) and with QA managers' knowledge. Although DTSs can provide useful data to facilitate many root-cause analyses, they can't necessarily substitute for human expertise in these analysis methods.

To improve the DTSs in companies PW and DP, we extended the existing defect attributes according to the company SQA and SPI goals. These moderate enhancements yielded quick and reliable insights to the quality and process issues of several company projects. Fewer defects and quicker fixes can yield benefits that go beyond lower maintenance costs, such as a better company reputation and bigger market share. We are continuing our work to collect more cost and benefit data of these DTS improvements to get a comprehensive understanding of their ROI.

References

1. M. Butcher, H. Munro, and T. Kratschmer, "Improving Software Testing via ODC: Three Case Studies," *IBM Systems J.*, vol. 41, no. 1, 2002, pp. 31–44.
2. V.R. Basili and H.D. Rombach, "The TAME Project: Towards Improvement-Oriented Software Environments," *IEEE Trans. Software Eng.*, vol. 16, no. 3, 1988, pp. 758–773.
3. F.V. Latum et al., "Adopting GQM-Based Measurement in an Industrial Environment," *IEEE Software*, vol. 15, no. 1, 1998, pp. 78–86.
4. R. Chillarege et al., "Orthogonal Defect Classification—A Concept for In-Process Measurements," *IEEE Trans. Software Eng.*, vol. 18, no. 1, 1992, pp. 943–956.
5. *IEEE Std. 1044-1993*, *Classification for Software Anomalies*, IEEE, 1994.
6. C. Jones, "Software Quality in 2008: A Survey of the State of the Art," slide presentation; www.scribd.com/doc/7758538/Capers-Jones-Software-Quality-in-2008, Software Quality Research LLC, 2008, pp. 37.
7. D. Rombach et al., "Impact of Research on Practice in the Field of Inspections, Reviews and Walkthroughs: Learning from Successful Industrial Uses," *ACM SIGSOFT Software Eng. Notes*, vol. 33, no. 18, 2008, pp. 26–35.
8. A.A. Shenvi, "Defect Prevention with Orthogonal Defect Classification," *Proc. India Software Eng. Conf.*, ACM, 2009, pp. 83–88.

Selected CS articles and columns are also available for free at http://ComputingNow.computer.org.

***IEEE SOFTWARE* CALL FOR PAPERS**

Special Issue on Technical Debt

SUBMISSION DEADLINE: 1 APRIL 2012 • PUBLICATION: NOVEMBER/DECEMBER 2012

The ability to deliver increasingly complex software-reliant systems demands better ways to manage the long-term effects of short-term expedient decisions. The idea of technical debt is that developers sometimes accept compromises in a system in one dimension (for example, modularity and code quality) to meet an urgent demand in another dimension (such as a deadline). Such compromises incur a "debt." Time spent dealing with the compromised code is considered "interest" that has to be paid, and the cost of building in the originally planned quality is the "principal" that should be repaid at some point for the long-term health of the project.

IEEE Software seeks submissions for a special issue on technical debt in software development. Possible topics include

- Definitions, models, or theories behind the concept of technical debt
- Case studies and lessons learned on technical debt in large-scale software development
- Practical guidelines, strategies, and frameworks for evaluating and paying back technical debt
- How to integrate technical debt management with software development practices (for example, Scrum, architecture analysis, design/code review and documentation, test-driven development, evolution, and maintenance)
- Approaches, applications, and tools for visualizing, analyzing, and managing technical debt
- Types, taxonomy, symptoms, and root causes of technical debt

QUESTIONS?

For more information about the special issue, contact the guest editors:

- Philippe Kruchten, University of British Columbia, Canada; pbk@ece.ubc.ca
- Robert L. Nord, Carnegie Mellon University, Software Engineering Institute; rn@sei.cmu.edu
- Ipek Ozkaya, Carnegie Mellon University, Software Engineering Institute; ozkaya@sei.cmu.edu

For full call for papers: www.computer.org/software/cfp6
For full author guidelines: www.computer.org/software/author.htm
For submission details: software@computer.org